

第三章 可行性研究

本章要点

- 一、可行性研究的任务
- 二、可行性研究的过程
- 三、可行性研究使用的工具
- 四、项目进度计划与成本分析

3.1 问题定义

- 问题定义就是描述问题，一般来说是用一句话来描述想做什么。
- 问题定义是整个软件工程的第一个步聚，甚至可以说软件工程里面各个项目的第一个步骤。
- 问题定义可能是来源于不同的领域，不同的方面。这种问题定义在现实生活中，可能有两个主要的来源，一个主要来源是做软件产品，另一个是软件的委托开发，不同的问题来源引起不同的问题定义。

问题定义报告的内容

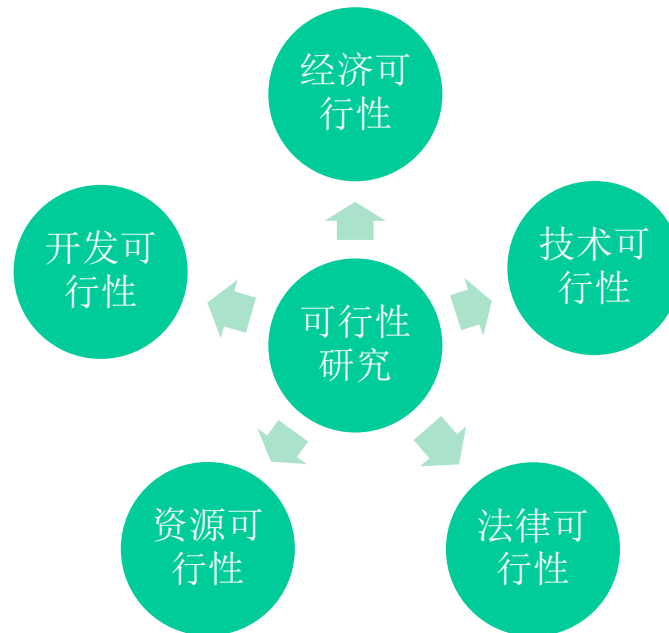
- 软件项目用途和目标。
- 软件项目类型和规模。
- 软件项目开发的开始时间以及大致交付使用的时间。
- 软件项目开发投入的经费。
- 软件的交付形式和内容。
- 是否需要培训及硬件。

3.2 可行性研究的任务

- 可行性研究的目的是用最小的代价在尽可能短的时间内确定问题是否能够解决。
- 关键：不是解决问题，而是确定问题是否值得去解。
- 研究在当前的具体条件下，是否具备开发系统必要的资源和其他条件。

可行性研究的任务

- 技术可行性
- 经济可行性
- 实施可行性
- 其他可行性
- 开发方案



不是解决问题，而是确定是否可解/值得解

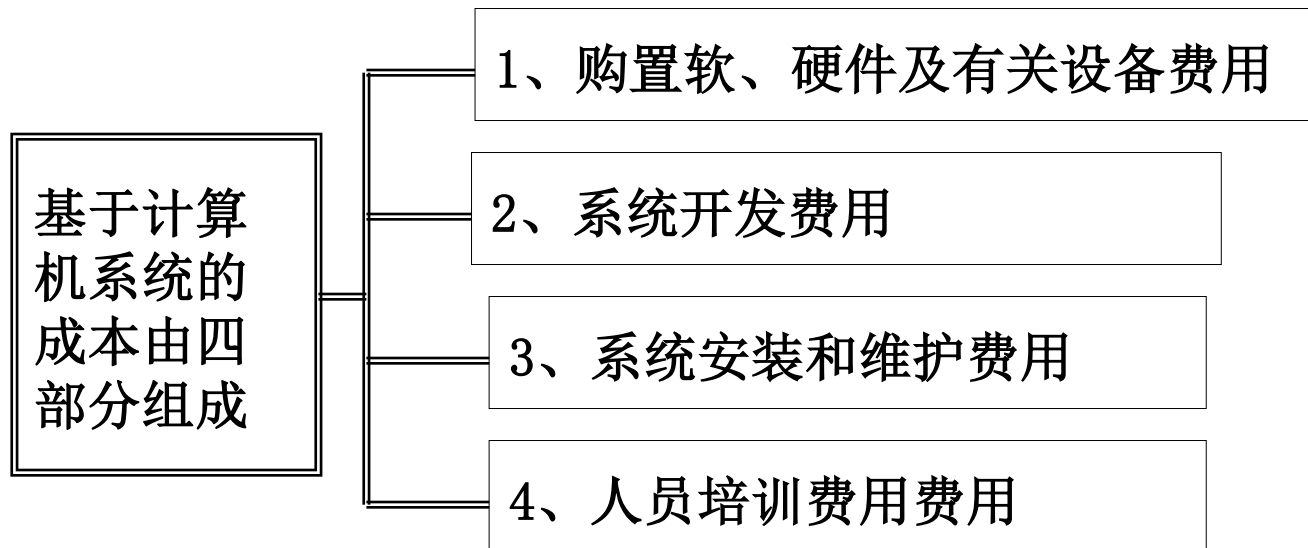
可行性研究的任务

1) 经济可行性:

经济可行性研究主要进行**成本效益分析**，包括估计项目的开发成本，估算开发成本是否会高于项目预期的全部利润。例如，构建电话服务中心，需配备两个电话接线员等工作人员，以及中继器、服务器等设备，因此，需调研一天打入的电话量，以确保系统运行后能获得利润，即在经济上是可行的。

可行性研究的任务

1) 经济可行性:



可行性研究的任务

2) 技术可行性:

- 技术可行性是最难决断并且最关键的问题。根据客户提出的系统功能、性能及实现系统的各项约束条件，从技术的角度研究系统实现的可行性。在此阶段，系统分析员应采集软件系统涉及到的各种信息，包括系统性能、可靠性、可维护性等方面，分析实现系统功能和性能所需要的各种设备、技术、方法和过程，并且需要分析软件开发在技术方面可能面临的风险，以及技术问题对开发成本的影响等。

可行性研究的任务

3) 资源可行性

- 资源可行性分析主要考虑软件开发组织是否具备开发该软件的资源，包括开发系统的人员是否存在问题，用于建立系统的其他资源，如硬件、软件等是否具备。

4) 法律可行性

法律可行性是指研究在系统开发过程中可能涉及的各种合同、侵权、责任以及各种与法律相抵触的问题。

可行性研究的任务

5) 开发可行性

提出系统实现的各种方案并进行评价之后，从中选择一种最优秀的方案。

- 根据成本、时间、人员、技术、设备等因素的不同要求，实现系统所提供的选择方案通常都不是唯一的。而每一种方案开发出来的系统在功能和性能方面都会存在很大的差异。
- 系统论证和选择、确定系统开发方案也是一个折衷过程。

3.3 可行性研究的步骤

1. 复查系统规模和目标

访问关键人员、仔细阅读和分析有关材料，复查问题定义中的规模和目标，明确目标系统的一切限制和约束。

2. 研究目前正在使用的系统

现有系统是信息的重要来源。新系统必须能完成旧系统的基本功能，也必须解决旧系统的缺陷。

了解相似的系统或软件。

3. 导出新系统的高层逻辑模型

从现有物理系统出发，导出现有系统的逻辑模型，设计新系统的逻辑模型，再导出物理模型。

可行性研究的步骤

4. 重新定义问题

复查问题定义，重新定义和修正问题，前四步骤是一个循环。

5. 导出和评价供选择的解法

从逻辑模型出发，导出几种解决方案。从技术可行性考虑排除不可行的方案，其次考虑实施可行性，估计开发成本和费用。最后，为每个技术、操作和经济都可行的系统制定实现进度表。

6. 推荐一个方案

根据可行性研究结果做出决定，是否可行，及给出可行的方案。

可行性研究的步骤

7. 草拟开发计划

- 1) 任务分解，确定负责人，项目小组
- 2) 大致进度规划
- 3) 财务预算
- 4) 风险分析及对策

技术风险、市场风险、政策风险

8. 书写文档提交审查

把上述可行性研究各个步骤的结果写成清晰的文档，并提交给用户和使用部门的负责人仔审查。

3.4 系统流程图

系统流程图

- 系统流程图作为概括描述物理系统的图形工具。
- 主要使用图形符号描述系统里的每个部件。
- 表现信息在系统各部件之间的流动，是物理数据流图，不是控制过程。

可行性研究阶段使用的工具

系统流程图-1基本符号

表 3-1 系统流程图的基本符号

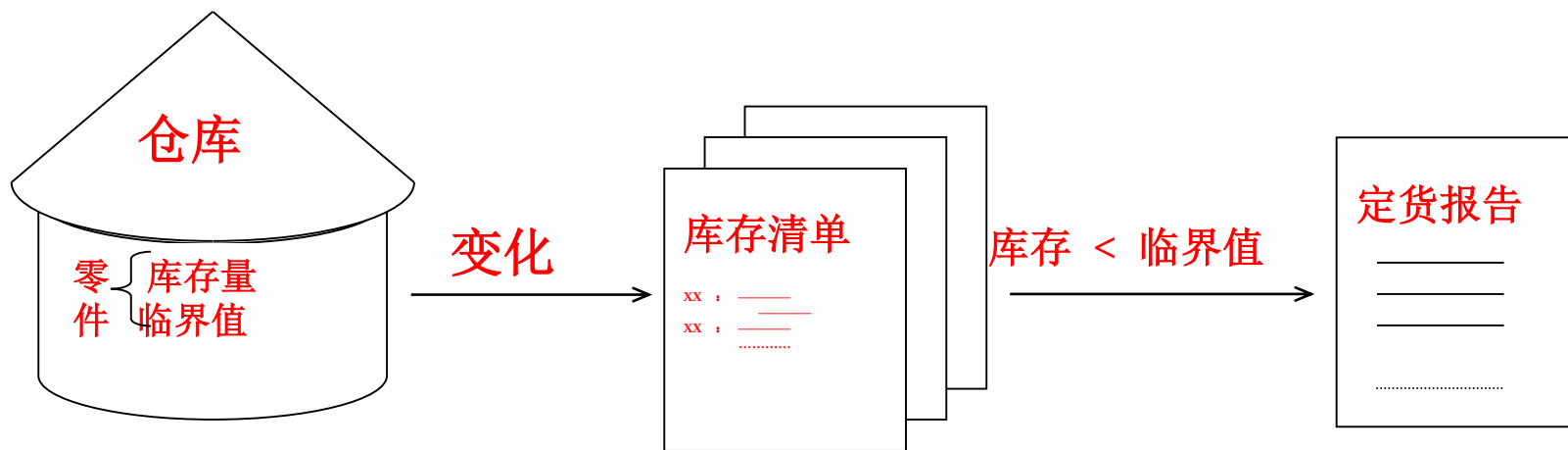
符号	名称	说明
	处理	能改变数据或数据位置的加工或部件
	输入/输出	表示输入/输出，是一个广义的不指名具体设备的符号
	连接	指出转到图的另一部分或从图的另一部分转来，通常在同一页上
	换页连接	指出转到另一页图上或从另一页图转来
	人工操作	由人工完成处理
	通信链路	远程通讯线路传输数据
	数据流	用来连接其他符号，指明数据流动方向

系统流程图的作用

1. 制作系统流程图的过程是系统分析员全面了解系统业务处理情况的过程，它是系统分析员作进一步分析的依据。
2. 系统流程图是系统分析员、管理人员、业务操作人员相互交流的工具。
3. 系统分析员可直接在系统流程图上拟出可以实现计算机处理的部分。
4. 可利用系统流程图来分析业务流程的合理性。

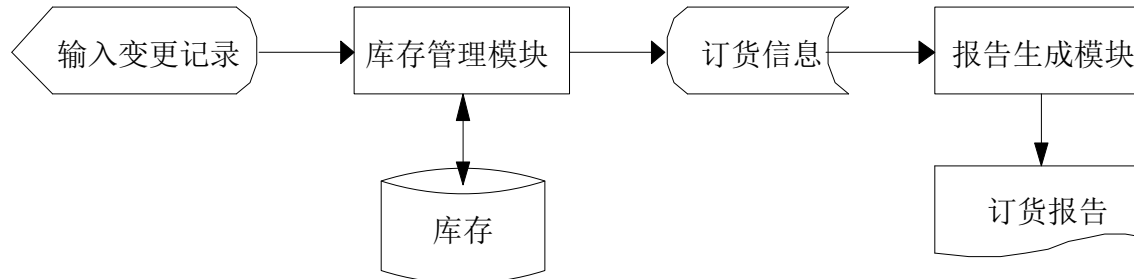
系统流程图—例题

例题1：某装配厂有一座存放零件的仓库，仓库中现有的各种零件的数量以及每种零件的库存量临界值等数据记录在库存清单主文件中。当仓库中零件数量有变化时，应该及时修改库存清单文件，如果哪种零件的库存量少于它的库存量临界值，则应该报告给采购部门以便定货，规定每天向采购部门送一次定货报告。



分析方法

- 分析：零件的变更由放在仓库中的终端输入到计算机中。
- 系统中库存管理程序对用户的输入做处理
- 计算机处理更新库存清单数据库和产生订货报告任务
- 更新库存清单数据库
- 每天报告生成程序打印定货报告

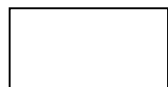


3.5 数据流图

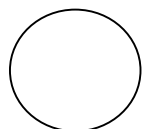
- 计算机已经演变为一个处理信息的工具，软件系统也是以信息处理为核心。在可行性研究阶段，最关键的是分析现在有什么数据，处理之后变成了什么数据，而至于计算机怎样处理则不是现阶段所关心的。
- 数据流图（DFD）是一种图形化技术，描绘信息和数据从输入移动到输出的过程所经历的变换。
- 数据流图中没有任何具体的物理部件，只是描绘数据在软件中流动和被处理的逻辑过程。
- 数据流图是系统逻辑功能的图形表示，是逻辑模型。

可行性研究阶段使用的工具

数据流图—符号



Input/output 数据源点/终点



function 变换数据的处理



Data storage 数据存储

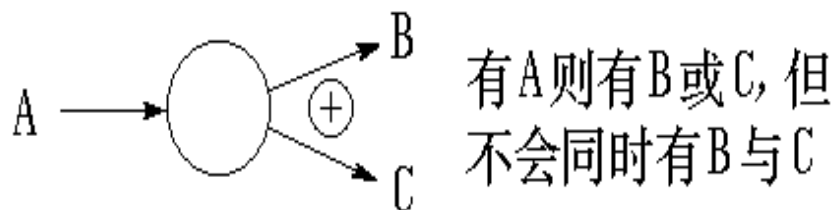
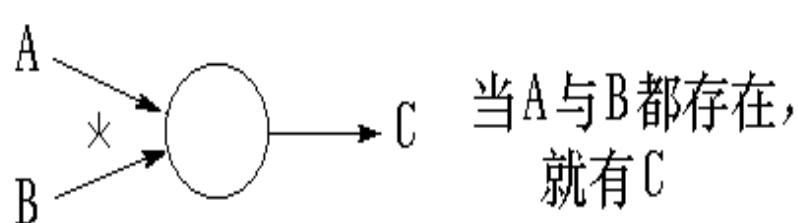
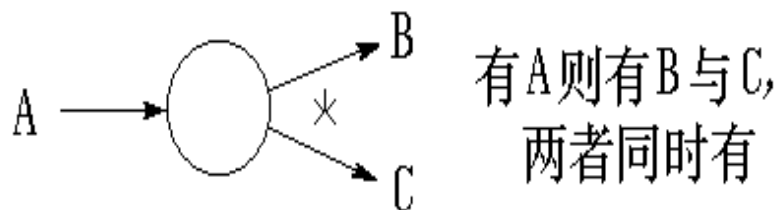
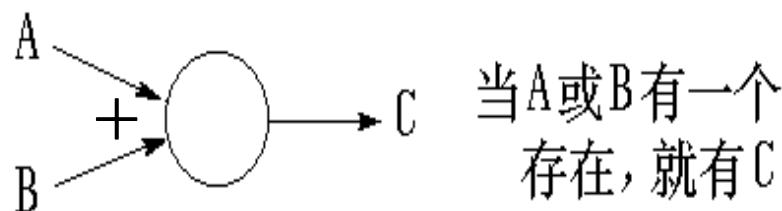
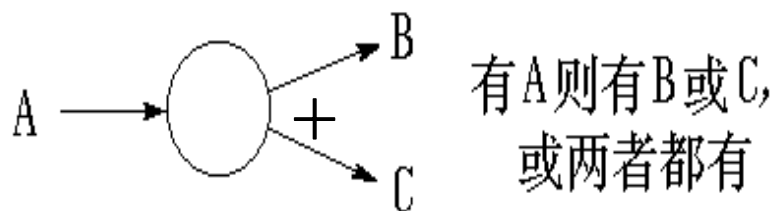


Data flow 数据流

- 处理并不代表一个程序，处理框可代表一系列程序
- 如果数据的源点和终点相同，不推荐使用一个符号
- 数据流图上忽略出错处理，以及打开或关闭文件等内务处理
- 描述“做什么”，不是怎么做
- *表示与，+表示或， $\oplus$ 表示只能选一个

可行性研究阶段使用的工具

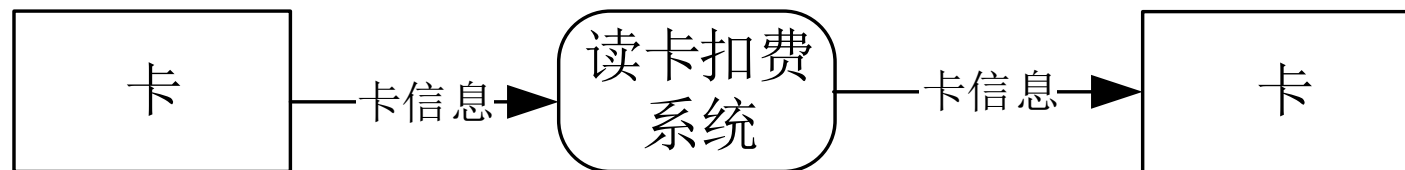
数据流图附加符号含义



例子

数据流图

公交车一卡通的顶层数据流图



2. DFD画法与命名

- 画DFD图时，一般遵循“由外向里”的原则，即先确定系统的边界或范围，再考虑系统的内部，先画加工的输入和输出，再画加工的内部。即：
 - (1) 识别系统的输入和输出。
 - (2) 从输入端至输出端画数据流和加工，并同时加上文件。
 - (3) 加工的分解“由外向里”进行分解。
 - (4) 数据流的命名，名字要确切，能反映整体。
 - (5) 各种符号布置要合理，分布均匀，尽量避免交叉线。
 - (6) 先考虑稳定态，后考虑瞬间态。如系统启动后在正常工作状态，稍后再考虑系统的启动和终止状态。

给数据流或数据存储命名

给数据流或数据存储命名时应注意：

- (1) 数据流或数据存储的名称应跟数据本身相关联；
- (2) 名字应代表整个数据流（或数据存储）的内容，而不是仅仅反映它的某些成分；
- (3) 不要使用空洞的、缺乏具体含义的名字，如“数据”、“信息”、“输入”之类；
- (4) 如果在为某个数据流（或数据存储）起名字时遇到了困难，则很可能是因为在对数据流图分解不恰当造成的，应该尝试进行重新分解。

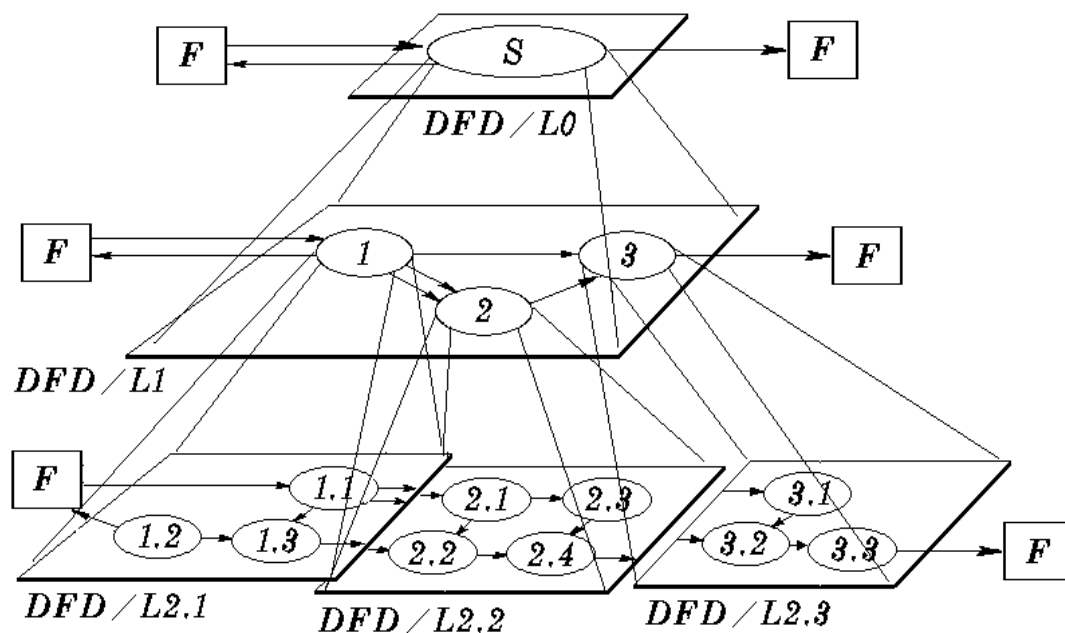
给处理命名

给处理命名重点描述数据是怎样处理的，注意以下原则：

- (1) 通常先为数据流命名，然后再为与之相关联的处理命名。这样命名比较容易，而且体现了人类习惯的“由表及里”的思考过程；
- (2) 名字应该反映整个处理的功能，而不是它的一部分功能；
- (3) 名字最好由一个具体的及物动词，加上一个具体的宾语组成。应该尽量避免使用“加工”、“处理”等空洞笼统的动词作名字；
- (4) 通常名字中仅包括一个动词，如果必须用两个动词才能描述整个处理的功能，则把这个处理再分解成两个处理可能更恰当些；
- (5) 如果在为某个处理命名时遇到困难，则很可能是发现了分解不当的情况，应考虑重新分解。

3 分层数据流图

为了表达数据处理过程的数据加工情况，需要采用**层次结构的数据流图**。按照系统的层次结构进行**逐步分解（处理）**，并以**分层**的数据流图反映这种结构关系，能清楚地表达和容易理解整个系统



分层数据流图

- ① 在多层数据流图中，顶层流图仅包含一个加工，它代表被开发系统。它的输入流是该系统的输入数据，输出流是系统所输出数据
- ② 底层数据流图是指其加工不需再做分解的数据流图，它处在最底层只要是模块的最底层就是底层流图，不管是第二层还是第三层的底层
- ③ 中间层流图则表示对其上层父图的细化。它的每一加工可能继续细化，形成子图。

分层数据流图

- ① 在多层数据流图中，顶层流图仅包含一个加工，它代表被开发系统。它的输入流是该系统的输入数据，输出流是系统所输出数据
- ② 底层数据流图是指其加工不需再做分解的数据流图，它处在最底层只要是模块的最底层就是底层流图，不管是第二层还是第三层的底层
- ③ 中间层流图则表示对其上层父图的细化。它的每一加工可能继续细化，形成子图。

画分层数据流图时应注意以下问题

1. 合理编号

- 分层数据流图的顶层称为0层,称它是第1层的父图,而第1层既是0层图的子图,又是第2层图的父图,依此类推。

2. 注意子图与父图的平衡

- 子图与父图的数据流必须平衡,这是分层数据流的重要性质。这里的平衡指的是子图的输入、输出数据流必须与父图中对应加工的输入、输出数据流相同。例如图3-5中3.1, 3.2, 3.3共有2个输入和1个输出,和3号加工的输入输出一致。

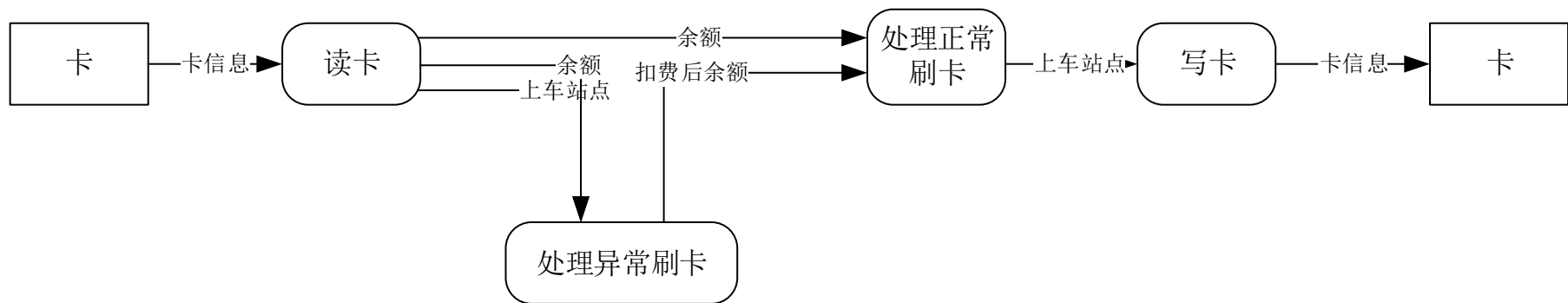
3. 分解的程度

- 对于规模较大的系统的分层数据流图,如果一下子把加工直接分解成基本加工单元,一张图上画出过多的加工将使人难以理解,也增加了分解的复杂度。

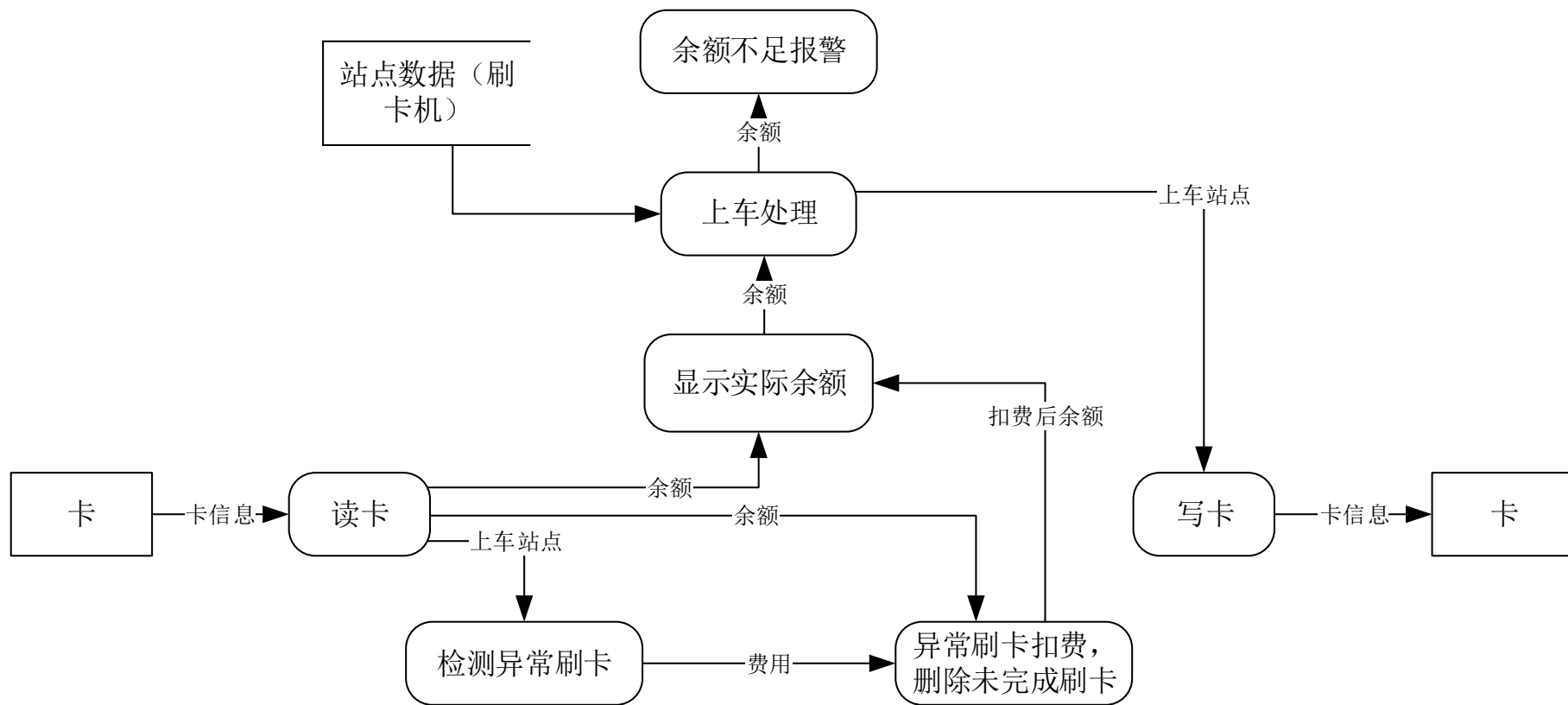
分解时应遵循以下原则

- 分解应自然，概念上要合理、清晰。
- 上层可分解得快些(即分解成的子加工个数多些)，这是因为上层是综合性描述，对可读性的影响小，而下层应分解得慢些。
- 在不影响可读性的前提下，应适当地多分解成几部分，以**减少分解层数**。
- 一般说来，当加工可用一页纸明确地表述时，或加工只有单一输入/输出数据流时(出错处理不包括在内)，就应停止对该加工的分解。另外，对数据流图中不再作分解的加工(即功能单元)，必须作出详细的加工说明，并且每个加工说明的编号必须与功能单元的编号一致。

公交一卡通上车刷卡的数据流图



公交一卡通上车刷卡的数据流图



检查数据流图

检查和修改数据流图的原则

- ① 数据流图上所有图形符号只限于前述四种基本图形元素，并且必须包括前述四种基本元素，缺一不可。
- ② 数据流图的主图上的数据流必须封闭在外部实体之间。
- ③ 每个加工至少有一个输入数据流和一个输出数据流。
- ④ 在数据流图中，需按层给加工框编号。编号表明该加工所处层次及上下层的亲子关系。
- ⑤ 规定任何一个数据流子图必须与它上一层的一个加工对应，两者的输入数据流和输出数据流必须一致。此即父图与子图的平衡。
- ⑥ 可以在数据流图中加入物质流，帮助用户理解数据流图。
- ⑦ 图上每个元素都必须有名字。
- ⑧ 数据流图中不可夹带控制流。

3.6 软件项目进度计划

3.6.1 进度管理与工作分解

- 进度是对执行的活动和里程碑所制定的工作计划日期表。
- 进度管理是为了确保项目按期完成所需要的管理过程。
- 时间是项目规划中灵活性最小的因素，因为项目时间通常不是项目组所能决定的，而是由用户和市场决定的。进度问题是产生项目冲突的主要原因，尤其在项目的后期。软件项目存在许多不确定因素，会造成软件项目延期。

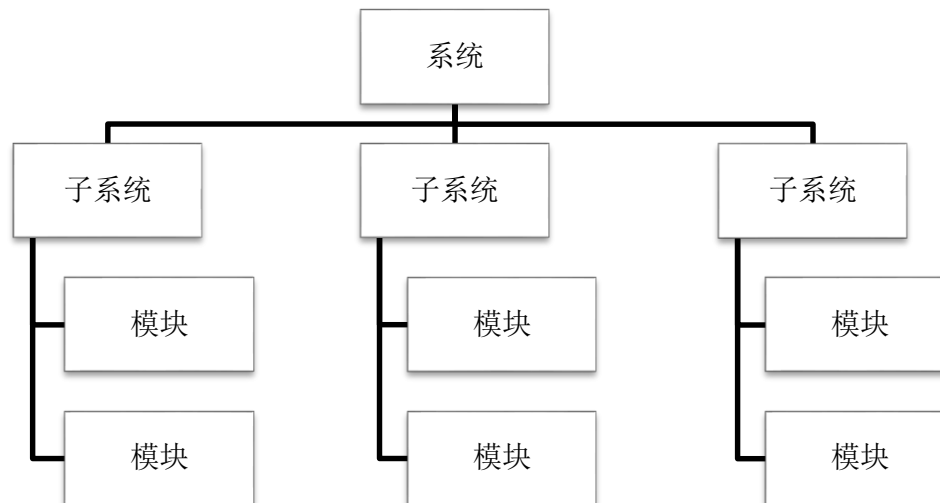
任务分解

- 要制定项目计划，就必须知道项目的任务和花费的时间。
- 什么是任务分解

将一个项目分解为更多的工作细目，使项目变得更易管理、更易操作。

- 可以采用WBS (Work Breakdown Structure) 工作分解结构。
- WBS是为了完成项目的目标和创造项目的可交付成果，由项目团队进行的一种对项目工作有层次的分解。

WBS的结构



WBS

- WBS包括以下概念：
 - (1) 工作包
- 工作包 (work package) 是WBS的最底层元素，一般的工作包是最小的“可交付成果”，这些可交付成果很容易识别出完成它的活动、成本和组织以及资源信息。
 - (2) WBS元素
- WBS元素实际上就是WBS结构上的一个个“节点”，通俗的理解就是“组织机构图”上的一个个“方框”，这些方框代表了独立的、具有隶属关系/汇总关系的“可交付成果”。

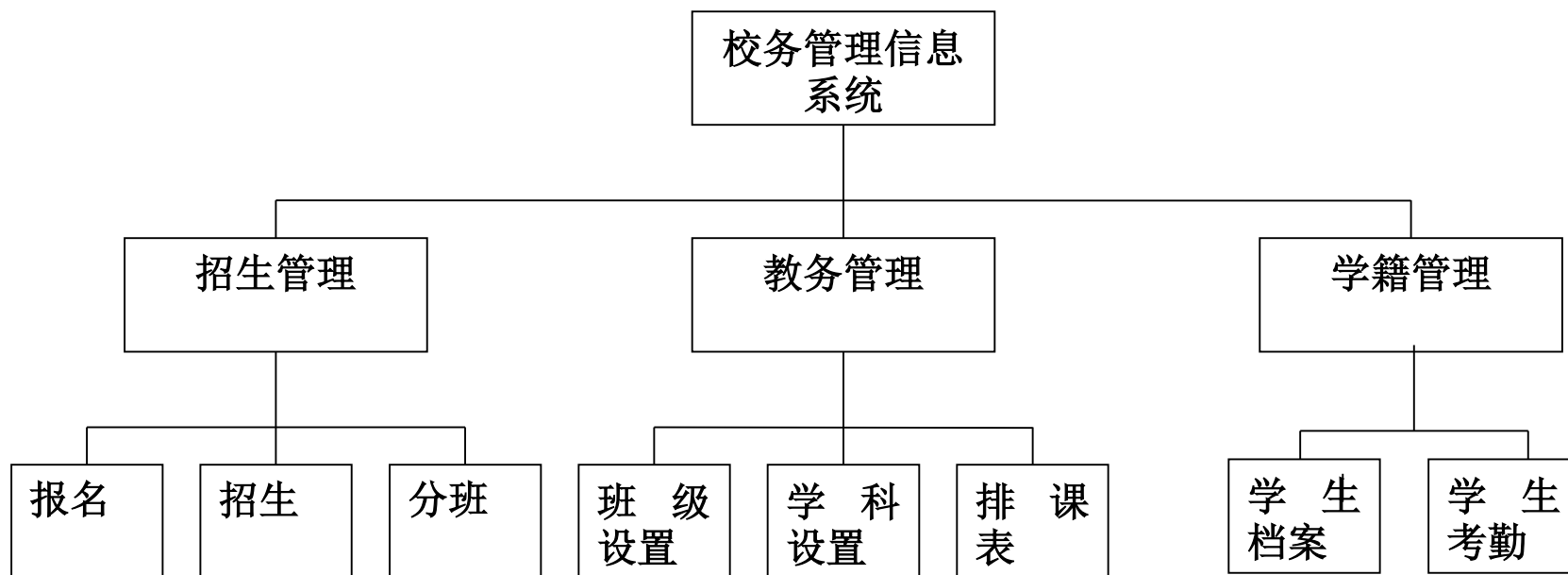
软件项目任务分解

WBS任务分解方式

- 1. 按产品或项目的功能分解。
 - 将软件系统按照子系统、功能、模块的方式进行分解。
 - 。
- 2. 按照实施过程分解。
 - 将软件系统按照需求分析、软件设计、编码、测试、维护进行分解。

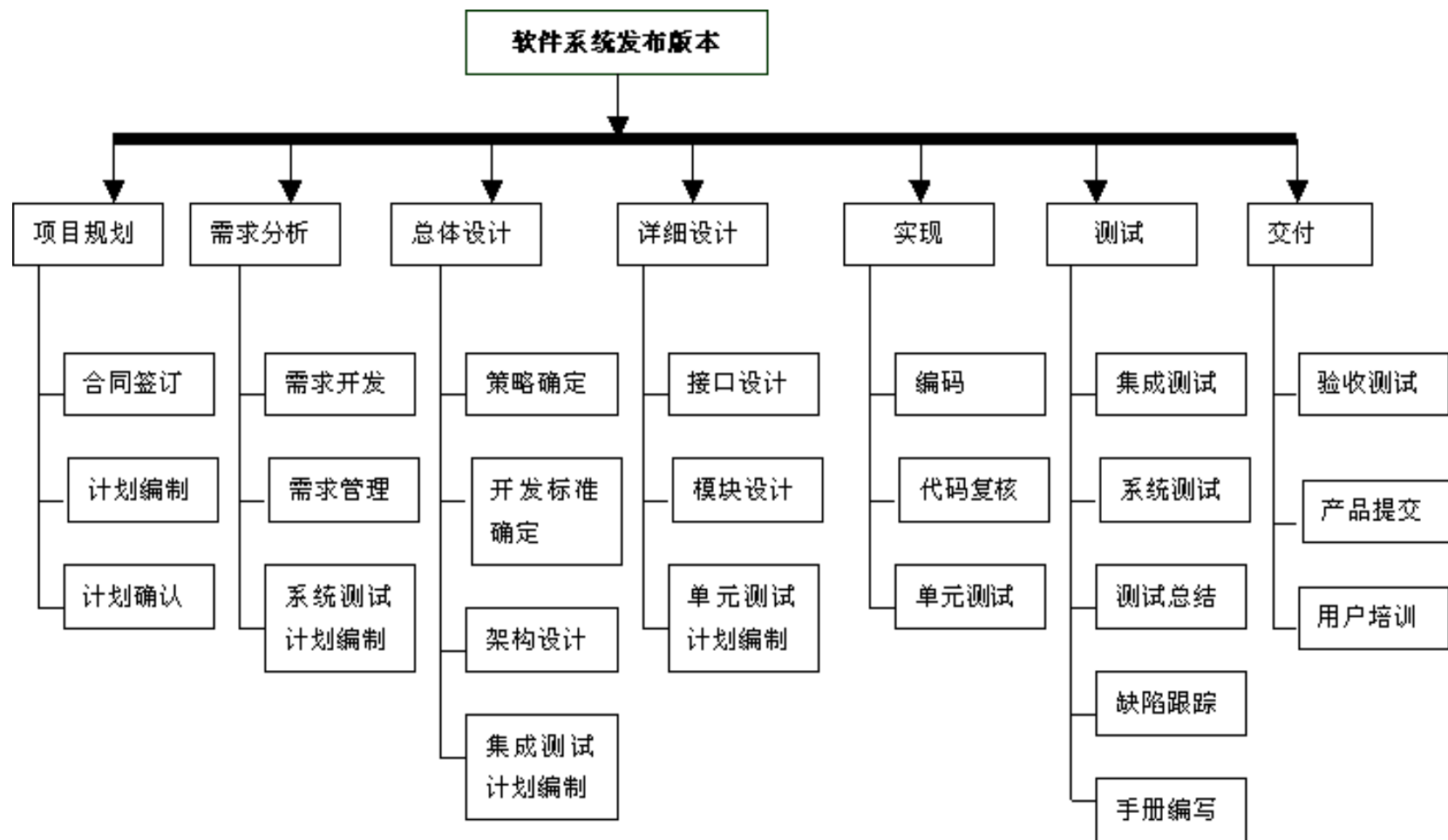
软件项目任务分解

按照产品的功能组成分解



软件项目任务分解

按照项目各阶段活动分解



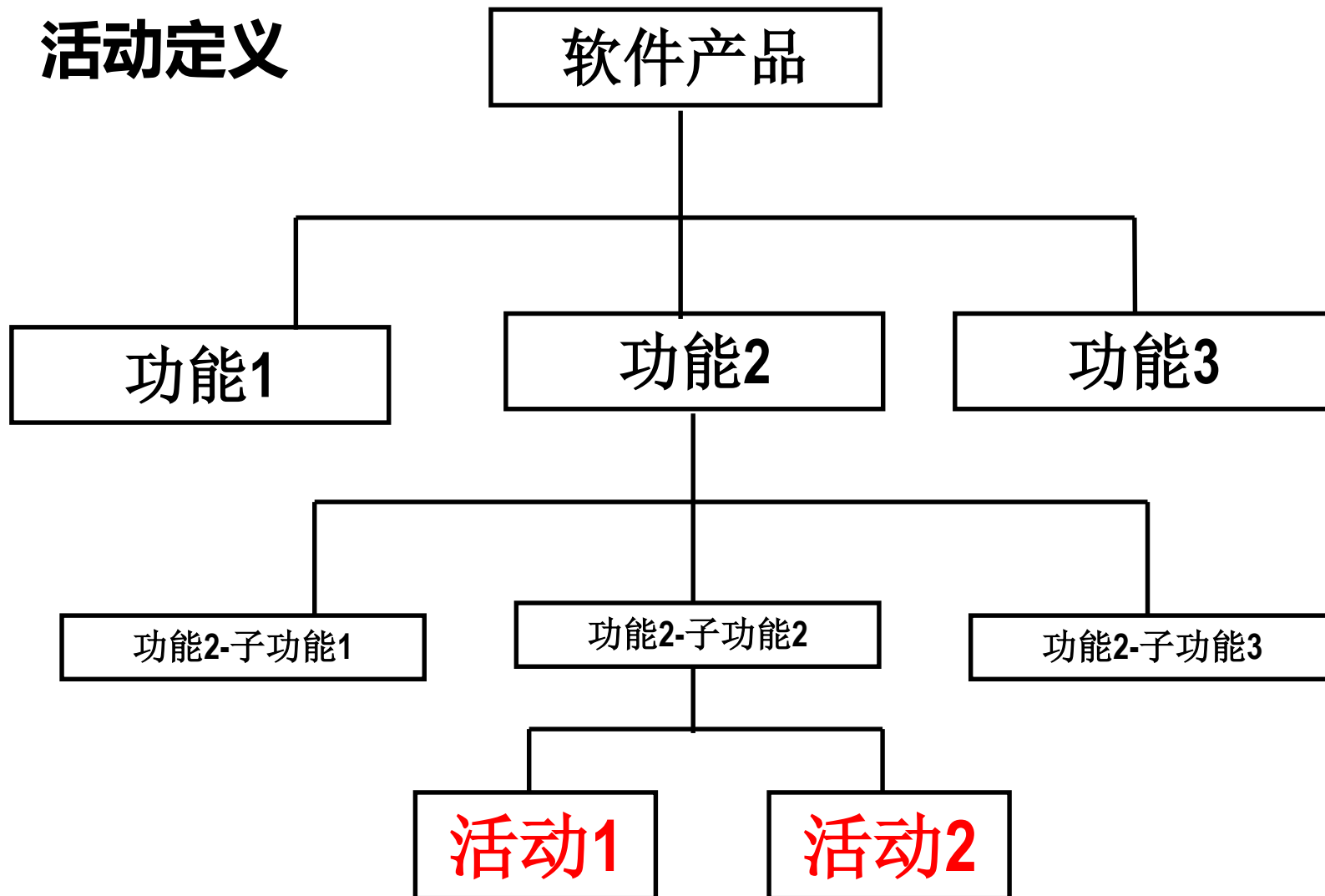
软件项目任务分解要求

- WBS的工作包的定义不超过40小时，但也不宜过细，建议在4-8小时。
- WBS的层次不超过10层，建议在4-6层。
- WBS中的支路没有必要全都分解到同一层次，即不必把结构强制做成对称的。在任意支路，当达到一个层次时，可以作出所要求准确性的估算，就可以停止了。
- 每个工作包要有一个交付成果。
- 对每个任务必须定义明确的完成标准。
- 必须有利于责任分配。

2 进度安排和进度图

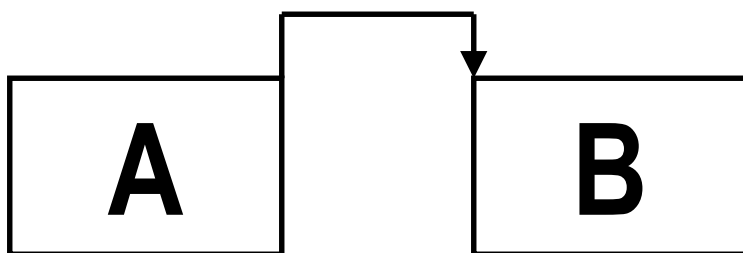
- 任务分解之后，要安排任务执行的时间顺序。
- 任务分解是面向可交付物的，因此需要对WBS做进一步分解，以便清楚为完成每个具体任务或交付物需执行哪些活动 (Activity) 。
- 活动指确定为完成项目的各个交付成果所必须进行的诸项具体活动。

活动定义

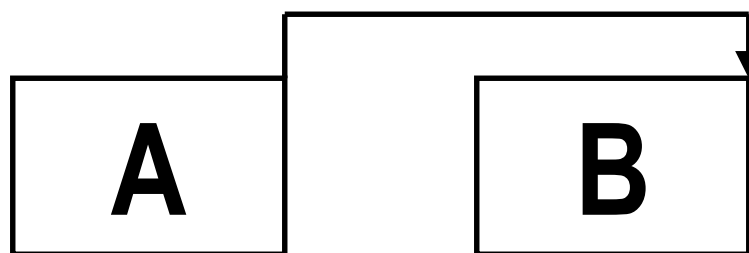


活动排序

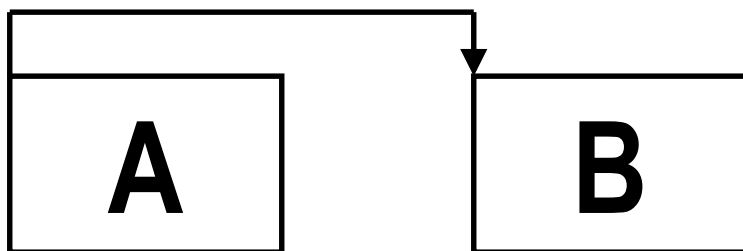
- 项目各项活动之间存在相互联系与相互依赖关系，要根据这些关系对活动进行适当的顺序安排。



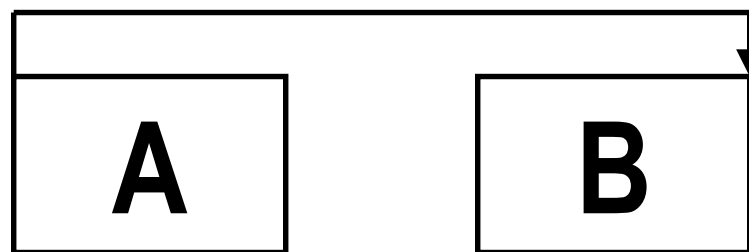
结束-开始 (FS)



结束-结束 (FF)



开始-开始 (SS)



开始-结束 (SF)

依赖关系

活动排序的依据

- 强制性依赖关系：工作任务中固有的依赖关系，它是因为客观规律和物质条件的限制造成的，又称硬逻辑关系。
- 软逻辑关系：由项目管理人员确定的项目活动之间的关系，它是一种根据主观判断去调整和确定的关系，也称指定性相关、偏好相关或软相关。
- 外部依赖关系：项目活动对一些非项目活动和事件的依赖。

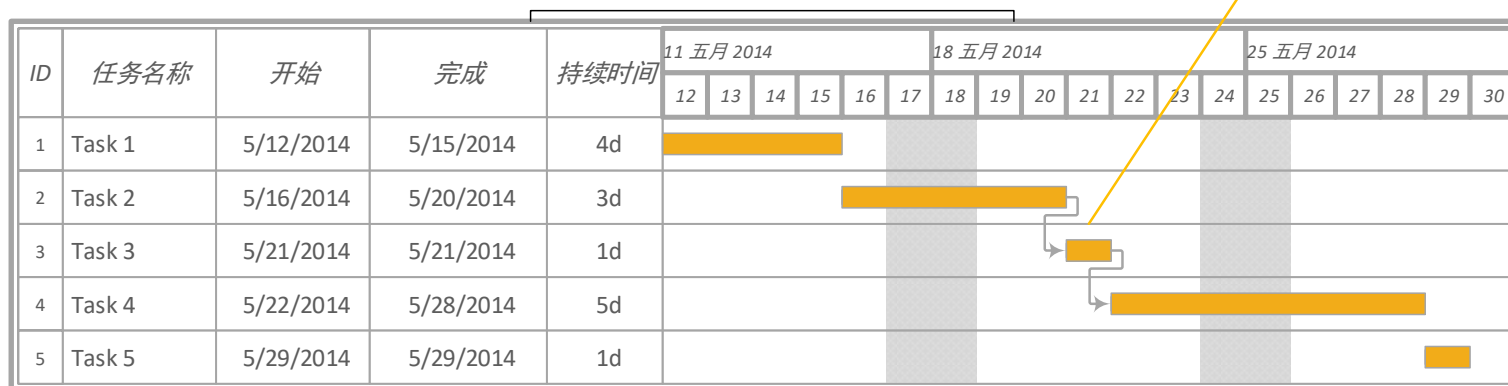
进度计划图

项目进度计划图

- 甘特图
- 网络图
- 里程碑图

甘特图

基本甘特图



甘特图的特点

- 可以查看活动的工期、开始时间和结束时间以及资源的信息，可用于详细的时间管理。
- 简单、直观、易于编制。
- 活动之间的依赖关系没有表示出来，难以进行定量的计算分析和计划的优化。

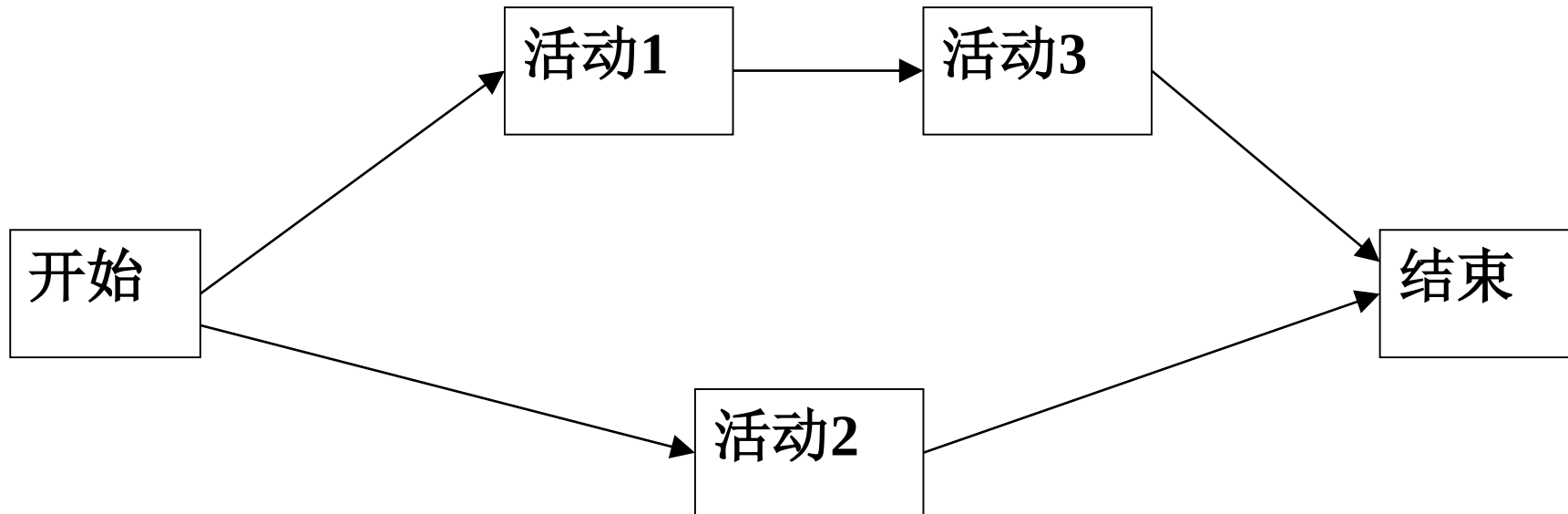
网络图

- 网络图是一种图解模型，形状如同网络，故称为网络图。网络图是由作业（箭线）、事件（又称节点）和路线三个因素组成的。网络图是用箭线和节点将某项工作的流程表示出来的图形。
- 网络图最常用的是前导图法（Precedence Diagramming Method，简称PDM）。PDM网络图的基本符号：
 - （1）节点：单代号网络图中每个节点表示一项工作，节点用圆圈或矩形表示；节点所表示的工作名称，持续时间和工作代号等应标注在节点内，单代号网络图中的节点必须

网络图

- (2) 箭线：单代号网络图中的箭线表示紧邻工作之间的逻辑关系，既不占用时间，也不消耗资源，箭线应画成水平的直线、折线或斜线。
- (3) 线路：单代号网络图中的，各条线路应用线路上的节点编号从小到大依次表述。

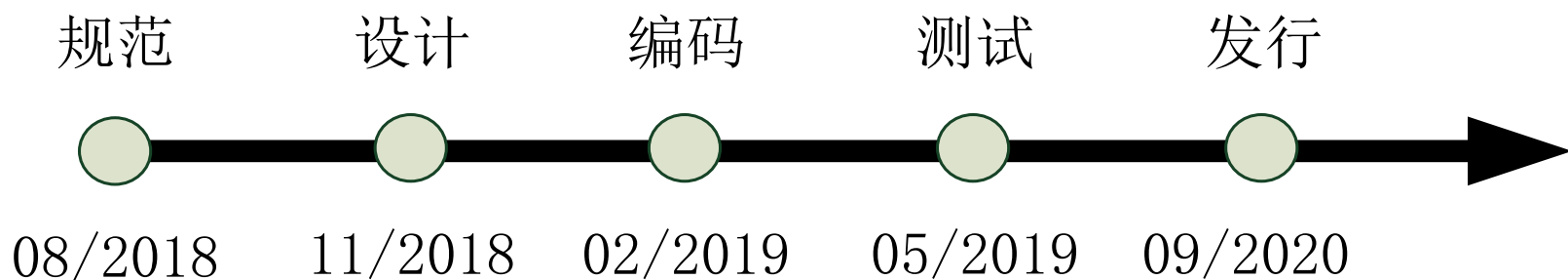
PDM图例



- 用节点(Box)表示活动。
- 用箭线表示各活动之间的逻辑关系。

里程碑图例

- 里程碑是项目进展中的重大事件，表示一个阶段的结束。里程碑图显示了项目中几个最重要的时间点，对项目管理者来说非常重要。



3 软件规模估计

- 软件规模估计是根据软件的开发内容、开发工具、开发人员等因素对需求分析、软件设计、编码、测试等整个开发过程所花费的时间和工作量所做的估计。
- 软件规模估算定义了软件的客观大小，而且不因为测量的人员、方式、时间的不同而变化。
- 做好软件项目管理的基础是要做好项目的规划工作，而做好项目规划的前提是要做好软件规模估计。

3 软件规模估计

软件规模的常用度量：**代码行** (Lines of Code, LOC) 和**功能点** (Function Point, FP)

1. LOC指所有的可执行的源代码行数。
2. 功能的规模体现在：用例数、功能点数、故事点数、页面数、窗口数、按钮数等。这其中，最得到普及应用的度量是功能点。

对软件规模常见的估算方法

1. 经验法。根据管理人员以往的项目或领域的经验，对未来的工作量进行估计。
2. 类推法
 - 将本项目的部分属性与高度类似的一个或几个完成的项目进行比对，适当调整后获得待估算项目的工作量、工期或成本估算值的方法。

对软件规模常见的估算方法

3. 方程法

减小估算误差的方法：

请多位专家分别估算程序的最小规模 a ，最可能的规模 m ，和最大规模 b ，再计算出这三种规模的平均值 $\bar{a}$ 、 $\bar{m}$ 和 $\bar{b}$ ，然后使用下式计算出程序规模的估计值：

$$L = \frac{\bar{a} + 4\bar{m} + \bar{b}}{6}$$

4. 交叉验证

- 估算过程中宜采用不同的方法分别估算，并进行交叉验证。如果不同方法的估算结果产生较大差异，可采用专家评审的方法确定估算结果，也可以使用简单的加权平均方法。

3.7 成本/效益分析

- 1. 代码行估计成本
- 根据代码行估计成本是比较简单的定量估算方法，它把开发每个软件功能的成本和实现这个功能需要用的源代码行数联系起来。通常根据经验和历史数据估计实现一个功能需要的源程序行数。
- 2. 任务分解估计成本
- 这种方法首先把软件开发工程分解为若干个相对独立的任务，再分别估计每个独立的开发任务的成本，最后加起来得出软件开发工程的总成本。

最常用按开发阶段划分任务，估计每个任务的成本，最后累加得出软件开发工程总成本，通常先估计完成该项任务需要的人力（人/月），再乘以每人每月平均工资得出每个任务成本。典型环境下各个开发阶段需要使用的人力的百分比如表。

任务	人力（%）
可行性研究	5
需求分析	15
设计	25
编码和单元测试	20
* 综合测试	35

2 效益分析

- 软件项目的效益可以分为有形效益和无形效益。有形效益一般指经济效益，无形效益主要是从性质上和心理上进行衡量，很难进行量的比较。但是无形效益有特殊的潜在价值，且在某些情况下会转化成有形的效益。
- 一般说来，经济效益通常表现为减少运行费用或增加收入。
- 经济效益可用投资回收期、纯收入等指标进行度量。

成本效益分析的方法

- ① 投资回收期。使累计的经济效益等于最初投资所需要的时间。回收期越短就越快获得利润
- ② 纯收入。整个生命周期内系统的累计经济效益（折合成现在值）与投资之差。